LIST OF EXPERIMENTS

1.Perform basic Image Handling and processing operations on the image is to read an image in python and Convert an Image to Gray-scale.

```
import cv2
import matplotlib.pyplot as plt
# Step 1: Read the image using OpenCV
image = cv2.imread('image.jpg')  # Replace with your image path
# Check if image is loaded
if image is None:
    print("Error: Image not found!")
else:
    print("Image loaded successfully")
    # Step 2: Convert BGR → RGB (for correct display)
    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    # Step 3: Convert to grayscale
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    # Step 4: Display images using matplotlib
    plt.figure(figsize=(8,4))
    plt.subplot(1,2,1)
    plt.imshow(image_rgb)
    plt.title("Original Image")
    plt.axis('off')

    plt.subplot(1,2,2)
    plt.imshow(gray, cmap='gray')
    plt.title("Grayscale Image")
    plt.axis('off')
```

```
plt.show()
# Step 5: Save grayscale image using matplotlib
plt.imsave('grayscale_matplotlib.png', gray, cmap='gray')
print("Grayscale image saved successfully using matplotlib")
```

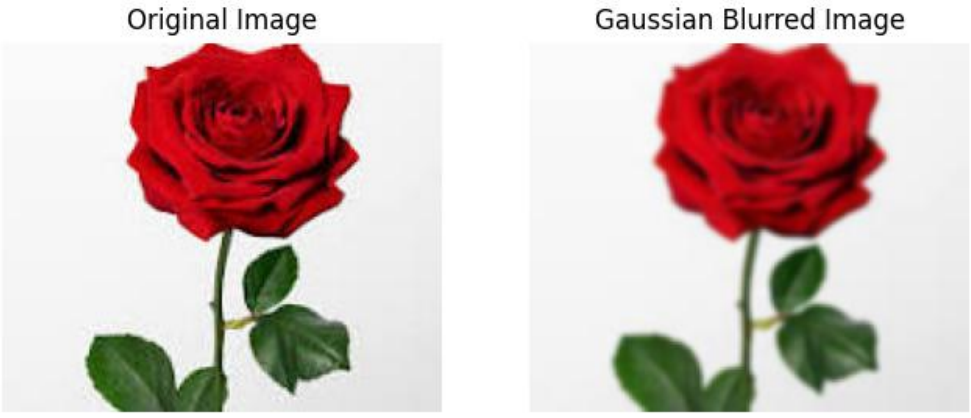
2. Perform basic Image Handling and processing operations on the image is to read an image in python and Convert an Image to Blur using Gaussian Blur

```
import cv2
import matplotlib.pyplot as plt

# Step 1: Read Image (Handling)
image = cv2.imread('image.jpg')  # Replace with your image path
if image is None:
    print("Error: Image not found!")
else:
    print("Image loaded successfully")
    # Step 2: Convert BGR → RGB
    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    # Step 3: Apply Gaussian Blur (Preprocessing)
    # (kernel size must be odd: (5,5), (7,7), etc.)
```

```
 blurred = cv2.GaussianBlur(image, (5, 5), 0)
    # Convert blurred image for display
    blurred_rgb = cv2.cvtColor(blurred, cv2.COLOR_BGR2RGB)
    # Step 4: Display Images
    plt.figure(figsize=(8,4))
    plt.subplot(1,2,1)
    plt.imshow(image_rgb)
    plt.title("Original Image")
    plt.axis('off')
    plt.subplot(1,2,2)
    plt.imshow(blurred_rgb)
    plt.title("Gaussian Blurred Image")
    plt.axis('off')
    plt.show()
    # Step 5: Save Blurred Image
    plt.imsave('gaussian_blur_output.png', blurred_rgb)
print("Blurred image saved successfully")
```

OUTPUT:-



3.Perform basic Image Handling and processing operations on the image is to read an image in python and Convert an Image to show outline using Canny function.

```
import cv2
import matplotlib.pyplot as plt
# Step 1: Read Image (Handling)
# ----------------------------
image = cv2.imread('image.jpg')  # Replace with your image path
if image is None:
    print("Error: Image not found!")
else:
    print("Image loaded successfully")
    # Step 2: Convert BGR → RGB
    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    # Step 3: Convert to Grayscale
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    # Step 4: Apply Canny Edge Detection
    # threshold1 = 100, threshold2 = 200
    edges = cv2.Canny(gray, 100, 200)
    # Step 5: Display Images
    plt.figure(figsize=(10,4))
    plt.subplot(1,3,1)
    plt.imshow(image_rgb)
    plt.title("Original Image")
    plt.axis('off')
    plt.subplot(1,3,2)
    plt.imshow(gray, cmap='gray')
    plt.title("Grayscale Image")
    plt.axis('off')
```

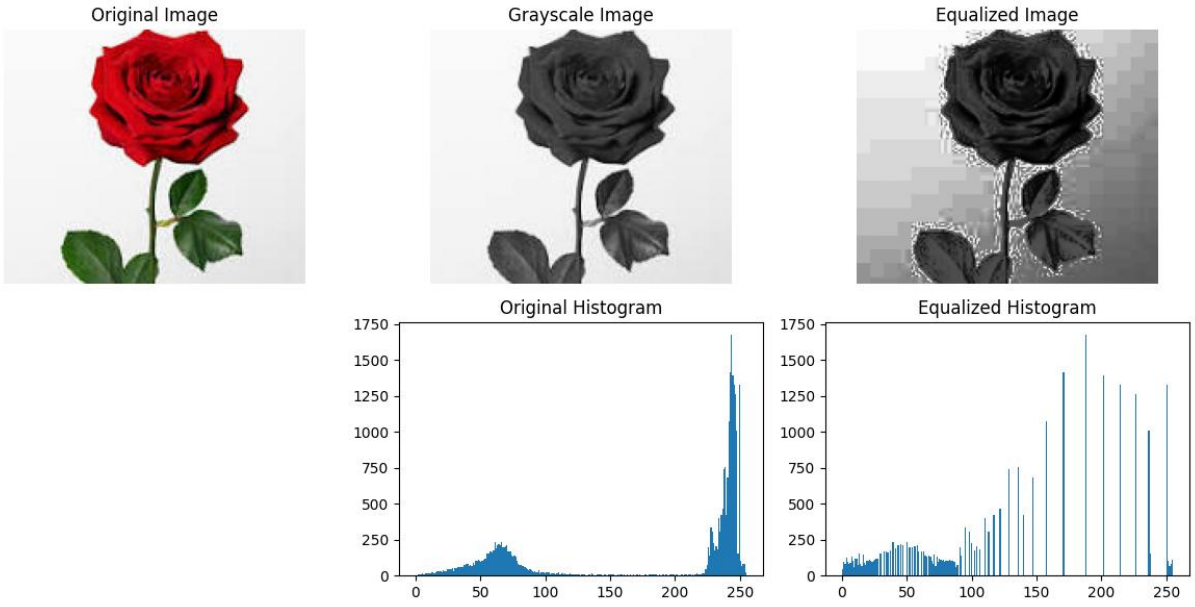
```
plt.subplot(1,3,3)
plt.imshow(edges, cmap='gray')
plt.title("Canny Edge Output")
plt.axis('off')
plt.show()
# Step 6: Save Edge Image
plt.imsave('canny_edges_output.png', edges, cmap='gray')
print("Edge-detected image saved successfully")
```

4.Implement histogram equalization on the given image and compare it with the original image using Open CV.

```
import cv2
import matplotlib.pyplot as plt
# Step 1: Read Image
image = cv2.imread('image.jpg')  # Replace with your image path
if image is None:
    print("Error: Image not found!")
else:
    print("Image loaded successfully")
```

```
# Step 2: Convert BGR → RGB (for display)
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
# Step 3: Convert to Grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
# Step 4: Histogram Equalization
equalized = cv2.equalizeHist(gray)
# Step 5: Display Images
plt.figure(figsize=(12,6))
plt.subplot(2,3,1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis('off')
plt.subplot(2,3,2)
plt.imshow(gray, cmap='gray')
plt.title("Grayscale Image")
plt.axis('off')
plt.subplot(2,3,3)
plt.imshow(equalized, cmap='gray')
plt.title("Equalized Image")
plt.axis('off')
# Step 6: Plot Histograms
plt.subplot(2,3,5)
plt.hist(gray.ravel(), bins=256)
plt.title("Original Histogram")
plt.subplot(2,3,6)
plt.hist(equalized.ravel(), bins=256)
plt.title("Equalized Histogram")
plt.tight_layout()
plt.show()
```

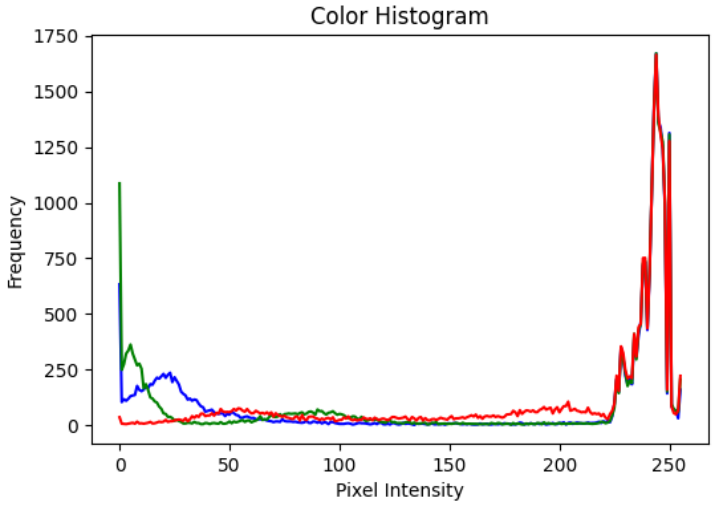
```
# Step 7: Save Images
plt.imsave('original_rgb.png', image_rgb)
plt.imsave('grayscale.png', gray, cmap='gray')
plt.imsave('equalized.png', equalized, cmap='gray')
print("All images saved successfully")
```



5.Write a Python function to analyze the histogram of the given input image based on color levels using Open CV.

```
import cv2
import matplotlib.pyplot as plt
def analyze_histogram(image_path):
    # Step 1: Read image
    image = cv2.imread(image_path)
    if image is None:
        print("Error: Image not found!")
        return
    # Step 2: Split into color channels (BGR)
    channels = cv2.split(image)
```

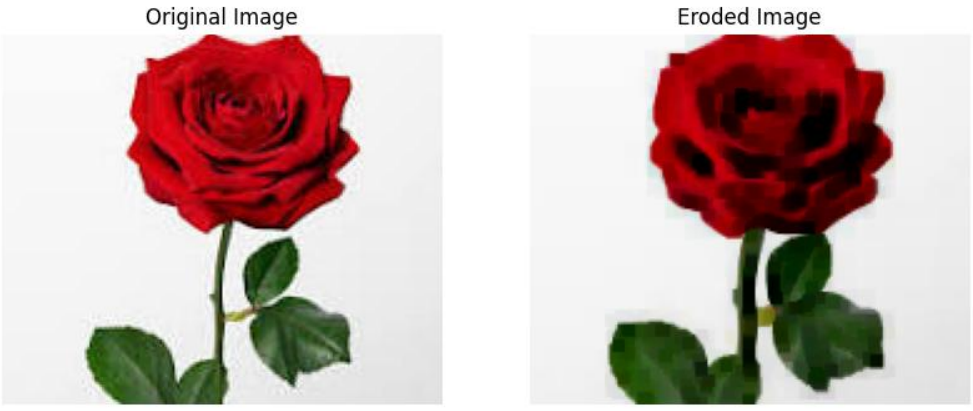
```
    colors = ('b', 'g', 'r')
    plt.figure(figsize=(6,4))
    plt.title("Color Histogram")
    plt.xlabel("Pixel Intensity")
    plt.ylabel("Frequency")
    # Step 3: Calculate histogram for each channel
    for channel, color in zip(channels, colors):
        hist = cv2.calcHist([channel], [0], None, [256], [0,256])
        plt.plot(hist, color=color)
    plt.show()
# Call the function
analyze_histogram('image.jpg')
```



6.Perform basic Image Handling and processing operations on the image is to read an image in python and Convert an Image to erode using Erode function.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# Read image
image = cv2.imread('image.jpg')
```

```
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
# Kernel
kernel = np.ones((5,5), np.uint8)
# Erosion
eroded = cv2.erode(image_rgb, kernel, iterations=1)
# Plot
plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis('off')
plt.subplot(1,2,2)
plt.imshow(eroded)
plt.title("Eroded Image")
plt.axis('off')
plt.show()
# Save
plt.imsave('eroded_output.png', eroded)
print("Eroded image saved successfully")
```



7.Perform basic video processing operations on the captured video. Read captured video in python and display the video, in slow motion and in fast motion.

8.Perform basic Image Handling and processing operations on the image is to read an image in python and Dilate an Image using Dilate function

9.Implement the image scaling techniques to resize the images to both bigger and small sizes

10.Perform a 90-degree rotation clockwise along the y-axis for the given image.

11 .Perform a 180-degree rotation clockwise along the y-axis for the given image

12.Perform a 270-degree rotation clockwise along the y-axis for the given image.

13.Perform Affine Transformation on the given image using python and Open CV

14.Perform Perspective Transformation on the given image using python and Open CV.

15.Perform basic Image Handling and processing operations on the image is to read an image in python and detect the corners in the image using Harris Corner Detection function

16.Implement a Sobel algorithm using Open CV to filter the input image.

17.Design and implement a water marking technique to insert the watermark into the original effectively image using Open CV.

18.Implement image cropping, copying and pasting to select a region of interest (ROI) from the source image using Open CV.

19.Implement the Erosion Morphological operations technique using Open CV in python.

20.Implement the Dilation technique as a Morphological operation to dilate the foreground regions based on Open CV.

21.Implement the Opening technique as a Morphological operation to dilate the foreground regions based on Open CV.

22.Implement the Closing technique as a Morphological operation to dilate the foreground regions based on Open CV.

23.Implement the Top hat technique as a Morphological operation to dilate the foreground regions based on Open CV.

24.Implement the Black hat technique as a Morphological operation to dilate the foreground regions based on Open CV.

25.Recognize watch from the given image by general Object recognition using Open CV.

26.Implement a function to reverse the frames of the video to create a video in reverse mode using Open CV.

27.Implement a face detection algorithm using Open CV to detect and locate human faces in the images.

28.Implement a vehicle detection algorithm using Open CV to detect and locate vehicles in each frame of the video.

29.Implement an Eye detection algorithm using Open CV to detect and locate human eyes in the images.

30.Implement a Smile detection algorithm using Open CV to detect and locate human smile in the images.

31.Implement a Segmentation algorithm using Open CV to segment the given input image based on the given threshold values.

32.Write a Python function to create a white image size entered by the user and then create 4 boxes of Black, Blue, Green and Red respectively on each corner of the image. The size of the colored boxes should be 1/10th the size of the image. (HINT: the arrays of ones and zeros can be in more than 2 dimensions).

33.Write a Python function to create a white image size entered by the user and then create a shape of Rectangle using Open CV.

34.Write a Python function to create a white image size entered by the user and then create a

shape of Circle using Open CV.

35.Write a Python function to create a text string entered by the user that must be appeared on the given image using Open CV.

36.Write a Python function to subtract the background of the given input image based on color levels using Open CV

37.Write a Python function to subtract the foreground of the given input image based on color levels using Open CV.

38.Write a Python function to Count the number of faces for the given input image using Open CV.

39.Write a Python function to play the given video in reverse mode in slow motion.

40.Write a Python function to extract the text from videos.